Deep Reinforcement Learning

Reinforcement Learning

Deep Reinforcement Learning

- Value-based deep RL: Deep-Q Networks, stabilization principles
- Policy gradient methods: direct policy search, policy gradient theorem, REINFORCE, baselines
- Actor-critic methods: Advantage Actor-Critic, Proximal Policy Optimization, Soft Actor-critic

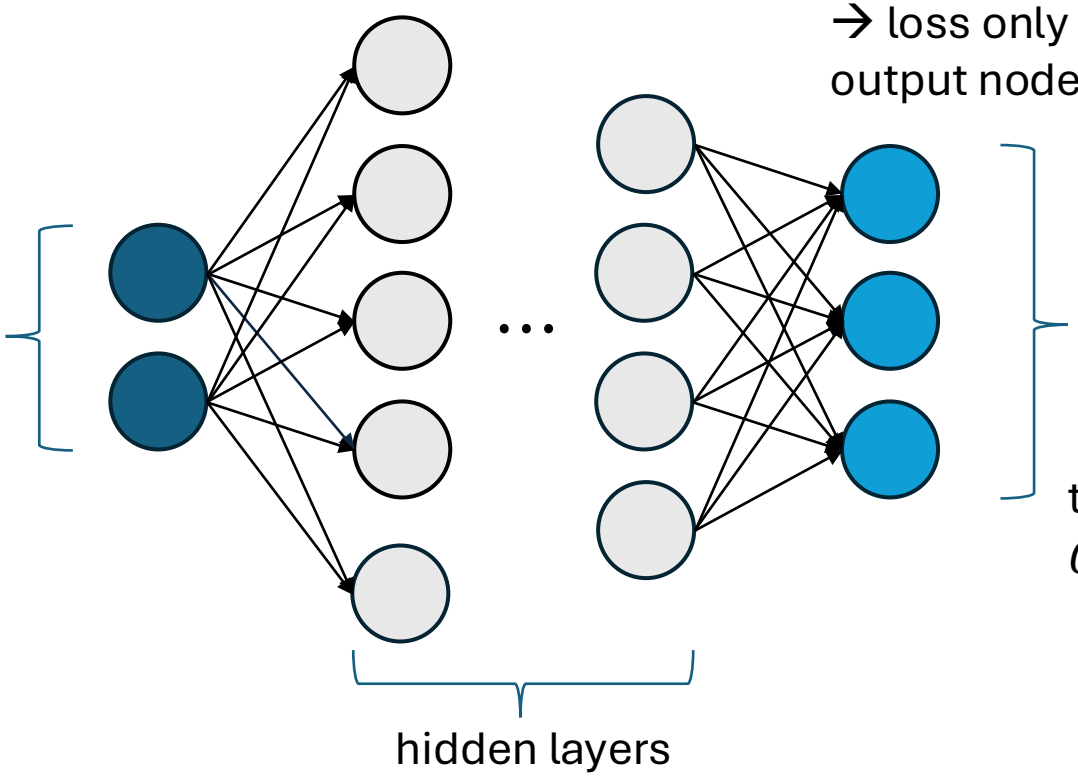
Value-Based Deep RL

Deep Q-Network (DQN)

idea: approximation of tabular Q-function by neural network

only one action for each training observation
→ loss only calculated from the corresponding output node (other outputs calculated but ignored)

values describing the state



Q-values for the different actions (→ discrete actions)

target:
$$Q(s_t, a_t) = r_{t+1} + \gamma \max_a Q(s_{t+1}, a_{t+1})$$

Distribution Shift in RL

the “deadly triad”:

bootstrapping + off-policy + function approximation

e.g., Q-learning

e.g., (deep) neural network

the data the network sees is nonstationary, leading to instability

Stabilization Principles

general principles for stabilizing value-based deep RL:

- replay buffers
- target networks
- clipping
- double Q
- prioritized replay

Key Components of DQN

two important tricks for training stabilization

experience replay:

training samples (or mini batches) drawn at random from previously stored experiences → remove correlations in observation sequence

separate target network:

copy of Q-network, but only periodically updated → reduce target correlations due to bootstrapping

Continuous Actions

instead of one output node for each discrete action, need to feed continuous action as input into the network

→ one output node for Q-value of the given action

problem: which action to choose then at runtime?

→ would need to run all possible actions through the network to find best Q-value

better: instead of Q-value, output action (i.e., policy) directly

Policy Gradient Methods

Direct Policy Search

goal: find an action policy without using value functions

parameterized policy $\pi(a_t|s_t;\boldsymbol{w})$:

probability to take different actions (target) given a state (variables/features) and parameters (e.g., neural network weights)

→ search for good parameters directly in the policy space using optimization algorithms

Objective

goal of policy search: maximizing expected return

→ objective function for parameterized policy $\pi(a_t|s_t;\boldsymbol{w})$:

$$J(\boldsymbol{w}) = \mathbb{E}_{\tau\sim\pi}[R(\tau)] = \int p_{\boldsymbol{w}}(\tau)\, R(\tau)\, d\tau$$

with trajectory $\tau = (s_0, a_0, s_1, a_1, \cdots)$,
return $R(\tau) = \sum_t \gamma^t r_t$,
and trajectory probability $p_{\boldsymbol{w}}(\tau) = \rho_0(s_0)\ \prod_t \pi(a_t|s_t;\boldsymbol{w})\ P(s_{t+1}|s_t, a_t)$

initial state distribution

Log-Likelihood Trick

using

$$\nabla_{\boldsymbol{w}}\, p_{\boldsymbol{w}}(\tau) = p_{\boldsymbol{w}}(\tau)\, \nabla_{\boldsymbol{w}} \log p_{\boldsymbol{w}}(\tau)$$

$$\nabla_{\boldsymbol{w}}\, J(\boldsymbol{w}) = \int \nabla_{\boldsymbol{w}}\, p_{\boldsymbol{w}}(\tau)\, R(\tau)\, d\tau = \int p_{\boldsymbol{w}}(\tau)\, \nabla_{\boldsymbol{w}} \log p_{\boldsymbol{w}}(\tau)\, R(\tau)\, d\tau$$

$$= \mathbb{E}_{\tau\sim\pi}[\underbrace{\nabla_{\boldsymbol{w}} \log p_{\boldsymbol{w}}(\tau)}\, R(\tau)] = \mathbb{E}_{\tau\sim\pi}\left[\left(\sum_t \nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})\right) R(\tau)\right]$$

$$\boxed{\begin{gathered}\log p_{\boldsymbol{w}}(\tau) = \log \rho_0(s_0) + \sum_t \log \pi(a_t|s_t;\boldsymbol{w}) + \sum_t \log P(s_{t+1}|s_t, a_t) \\ \Rightarrow \nabla_{\boldsymbol{w}} \log p_{\boldsymbol{w}}(\tau) = \sum_t \nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})\end{gathered}}$$

Policy Gradient Theorem

replace the full return $R(\tau)$ of a trajectory τ with the future return $G_t = \sum_{t'=t+1}^{\infty} \gamma^{t'-t} r_{t'}$ for each time step t:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \mathbb{E}_{\pi}\left[\sum_{t} \nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})\, G_t\right]$$

- $R(\tau) = G_t + \overbrace{(\text{past rewards until } t)}^{\text{independent of current action}}$
- $\mathbb{E}_{a\sim\pi}[\nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})] = 0$

→ past terms vanishes

→ gradient of an agent's performance with respect to its policy parameters, enabling policy optimization through gradient ascent

Side Note: Score Function Property

$$\mathbb{E}_{a_t \sim \pi}[\nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})]$$
$$= \sum_a \pi(a_t|s_t;\boldsymbol{w}) \nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w}) = \sum_a \nabla_{\boldsymbol{w}} \pi(a_t|s_t;\boldsymbol{w}) = 0$$

$$\nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w}) = \frac{\nabla_{\boldsymbol{w}} \pi(a_t|s_t;\boldsymbol{w})}{\pi(a_t|s_t;\boldsymbol{w})}$$

$$\sum_a \pi(a_t|s_t;\boldsymbol{w}) = 1$$
$$\Rightarrow \nabla_{\boldsymbol{w}} \sum_a \pi(a_t|s_t;\boldsymbol{w}) = 0$$
$$\Rightarrow \sum_a \nabla_{\boldsymbol{w}} \pi(a_t|s_t;\boldsymbol{w}) = 0$$

REINFORCE

approximate expectation in policy gradient theorem using MC samples:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{t} \nabla_{\boldsymbol{w}} \log \pi \left(a_t^{(i)} | s_t^{(i)}; \boldsymbol{w} \right) G_t^{(i)}$$

batch of full episodes

"weighting" with observed (discounted) return

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \cdot \nabla_{\boldsymbol{w}} J(\boldsymbol{w})$$

update after each episode (SGD, $n = 1$)
or after several ones (mini-batch gradient descent)

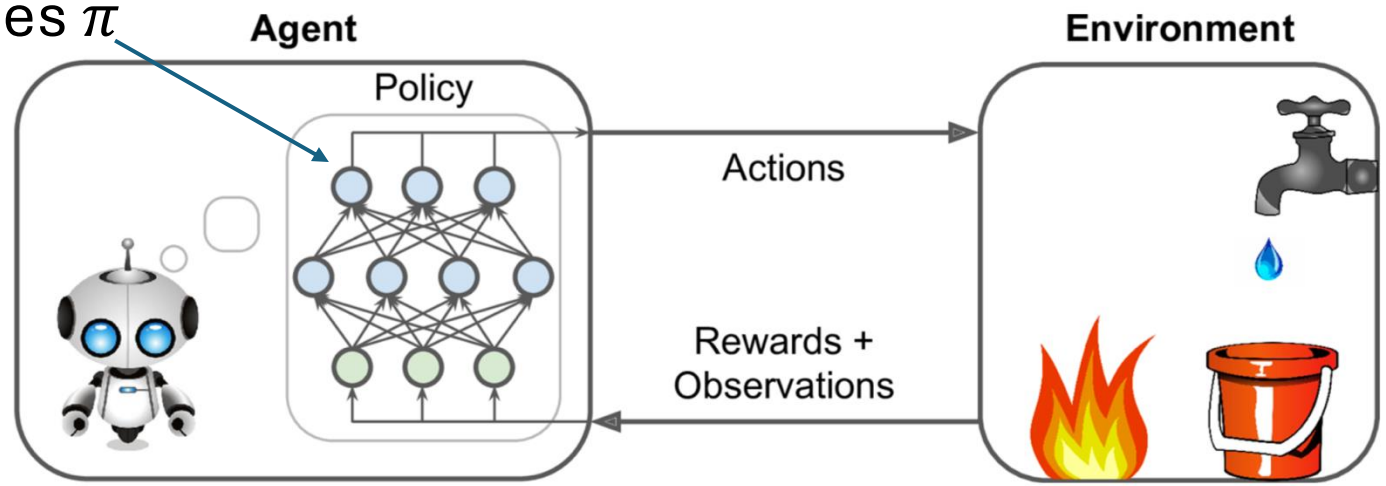
typical implementation: policy gradients → neural network gradients

Implementation of Policy Gradient Method

neural network with actions as output/target: policy network

action probabilities π

$\log \pi$: score function



observed returns
(MC sampling)

$$\boxed{\text{loss: } -\sum_t \log \pi(a_t|s_t;\boldsymbol{w})\, G_t}$$

gradient ascent
→ actions with higher rewards

choose action according to probability
→ implicit exploration, on-policy method

REINFORCE with Baseline

again: $\mathbb{E}_{a\sim\pi}[\nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})] = 0$

expectation unchanged when subtracting an action-independent baseline:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \mathbb{E}_{\pi}\left[\sum_t \nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w}) \left(G_t - b(s_t)\right)\right]$$

reduces variance if baseline is correlated with return:

if $b(s_t)$ close to G_t → smaller variance in $G_t - b(s_t)$ → smaller variance in gradient

example for baseline: average discounted return

Learned Value Function as Baseline

choose an estimate of the true state-value function as baseline:

$$b(s_t) = v(s_t|\boldsymbol{w}_v)$$

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \cdot \sum_t \nabla_{\boldsymbol{w}_\pi} \log \pi(a_t|s_t; \boldsymbol{w}_\pi) \left(G_t - v(s_t|\boldsymbol{w}_v)\right)$$

e.g., separate networks
trained using MC returns

Actor-Critic Methods

Actor-Critic

TD error

use bootstrapping:

$$G_t - v(s_t|\boldsymbol{w_v}) \approx r_t + \gamma v(s_{t+1}|\boldsymbol{w_v}) - v(s_t|\boldsymbol{w_v}) = \delta_t$$

→ $v(s_t|\boldsymbol{w_v})$ is trained via TD learning

$\pi(a_t|s_t;\boldsymbol{w}_\pi)$: actor
$v(s_t|\boldsymbol{w_v})$: critic

update of $\pi(a_t|s_t;\boldsymbol{w}_\pi)$:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \nabla_{\boldsymbol{w}_\pi} \log \pi(a_t|s_t;\boldsymbol{w}_\pi)\, \delta_t$$

→ policy updates can happen every time step

actor-critic = REINFORCE + learned baseline + bootstrapping
→ turning MC into TD method

State-Action Form of Policy Gradient Theorem

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \mathbb{E}_{\pi}\left[\sum_{t} \nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})\, G_t\right]$$

$$= \mathbb{E}_{s,a\sim\pi}[\nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})\, Q_{\pi}(s_t,a_t)]$$

using
$Q_{\pi}(s_t,a_t) = \mathbb{E}_{\pi}[G_t|s_t,a_t]$

→ actions are weighted by their expected returns

theoretical foundation: exact gradient identity that actor-critic methods approximate

Advantage Actor-Critic

advantage function: $A(s_t, a_t) = Q(s_t, a_t) - v(s_t)$

idea: calculates extra reward for specific action compared to average action in given state (how much better than expected)

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \mathbb{E}[\nabla_{\boldsymbol{w}} \log \pi(a_t | s_t; \boldsymbol{w})\, A(s_t, a_t)]$$

expectation unchanged when subtracting an action-independent baseline

Advantage Actor-Critic (A2C / A3C)

advantage estimate: $A(s_t, a_t) \approx \delta_t$

→ same gradients as before (but different interpretation as advantage)

actor update: $\boldsymbol{w}_\pi \leftarrow \boldsymbol{w}_\pi + \eta \cdot \delta_t \nabla_{\boldsymbol{w}_\pi} \log \pi(a_t | s_t; \boldsymbol{w}_\pi)$

critic update: $\boldsymbol{w}_v \leftarrow \boldsymbol{w}_v + \eta_v \cdot \delta_t \nabla_{\boldsymbol{w}_v} v(s_t | \boldsymbol{w}_v)$

$$\text{target} = r_t + \gamma v(s_{t+1} | \boldsymbol{w}_v)$$

$$L(\boldsymbol{w}_v) = \frac{1}{2}\Big(v(s_t | \boldsymbol{w}_v) - \big(r_t + \gamma v(s_{t+1} | \boldsymbol{w}_v)\big)\Big)^2 = \frac{1}{2}{\delta_t}^2$$

$$\delta_t = r_t + \gamma v(s_{t+1} | \boldsymbol{w}_v) - v(s_t | \boldsymbol{w}_v)$$

$$\nabla_{\boldsymbol{w}_v} \delta_t \approx -\nabla_{\boldsymbol{w}_v} v(s_t | \boldsymbol{w}_v)$$

(semi-gradient method: treat target as constant)

A2C vs A3C

A2C (Advantage Actor-Critic)

- synchronous updates
- uses batch of parallel environments
- more stable, GPU-friendly

A3C (Asynchronous Advantage Actor-Critic)

- multiple workers update independently
- no locking → faster but noisier
- CPU-friendly

- on-policy
- low variance (compared to REINFORCE)
- step-wise updates (fast learning)
- still sample inefficient
- can be unstable with function approximation

Proximal Policy Optimization (PPO)

stable and efficient policy gradient (actor-critic) method

main problem with vanilla policy gradient (and also A2C): large updates → policy collapse, unstable training

PPO ≈ A2C
+ (better advantage estimation)
+ (safer policy updates)
+ (multiple epochs on same data)

Generalized Advantage Estimation (GAE)

PPO uses GAE: a discounted sum of TD errors that interpolates between TD and Monte Carlo

A2C: 1-step advantage estimate: $A_t \approx \delta_t$

multi-step advantage estimate: $A_t^{(k)} \approx \delta_t + \gamma\delta_{t+1} + \cdots + \gamma^{k-1}\delta_{t+k-1}$

- larger k → closer to Monte Carlo
- smaller k → more bootstrapping

GAE combines all these with exponential weighting:

$$A_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}$$

equivalent form with λ-return $\hat{G}_t^{(\lambda)}$: $A_t = \hat{G}_t^{(\lambda)} - V(s_t)$

$\lambda = 0$: pure TD, $\lambda = 1$: MC, PPO typically $\lambda \approx 0.95$

→ lower variance, smoother learning

advantages usually normalized to improve stability:

$$A_t \leftarrow \frac{A_t - \mathrm{mean}(a)}{\mathrm{std}(A)}$$

Surrogate Objective

vanilla policy gradient (and A2C):

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \mathbb{E}_{\pi_{\boldsymbol{w}}}[\nabla_{\boldsymbol{w}} \log \pi(a_t|s_t;\boldsymbol{w})\, A(s_t, a_t)]$$

proxy objective (standard policy gradient surrogate):

$$L_{\mathrm{PG}} = \mathbb{E}_{\pi_{\mathrm{old}}}[r_t(\boldsymbol{w}_\pi) A(s_t, a_t)]$$

data from old policy

its gradient matches the true policy gradient at the current policy

using importance sampling to correct for mismatch (importance ratio r_t):

$$r_t(\boldsymbol{w}_\pi) = \frac{\pi(a_t|s_t;\boldsymbol{w}_\pi)}{\pi_{\mathrm{old}}(a_t|s_t;\boldsymbol{w}_{\mathrm{old}})}$$

likelihood ratio

problem: $r_t(\boldsymbol{w}_\pi) A(s_t, a_t)$ can become very large → unstable updates

Clipping

key idea of PPO: constrained updates by means of clipped surrogate objective

clipping: instead of changing the policy arbitrarily, PPO restricts the update to keep the new policy "proximal" (close) to the old one

$$L_{\mathrm{PPO}} = \mathbb{E}_{\pi_{\mathrm{old}}}\left[\min\left(r_t(\boldsymbol{w}_\pi)A(s_t, a_t), \mathrm{clip}(r_t(\boldsymbol{w}_\pi), 1-\epsilon, 1+\epsilon)\, A(s_t, a_t)\right)\right]$$

- if update is small: normal policy gradient
- if update is large: clipped → prevents destructive updates

Data Reuse

on-policy: collect data from π_{old}

A2C: use data only once (1 optimization epoch), then it's outdated
policy changes → gradients become incorrect → training becomes unstable
→ extremely sample-inefficient

PPO: data reuse (multiple optimization epochs)
possible because of clipped surrogate objective (controls policy drift)

PPO Algorithm

1. collect trajectories with π_{old}
2. compute advantages (often with GAE)
3. optimize clipped objective for several epochs
4. update policy

- stable training, simple to implement
- works well with neural networks
- strong empirical performance

- still on-policy (with limited off-policy optimization) → sample inefficient
- needs careful tuning (clip range, epochs)

Soft Actor-Critic (SAC)

in PPO, an entropy bonus can be added to the objective to encourage exploration

entropy: $\mathcal{H}(\pi) = -\mathbb{E}[\log \pi(a|s)]$

in SAC, entropy is part of the objective itself (maximum entropy actor-critic):

$$J(\pi) = \mathbb{E}\left[\sum_t r_t + \alpha\, \mathcal{H}\big(\pi(\cdot\,|s_t)\big)\right]$$

→ maximize reward and entropy

modified reward: $r_{\text{soft}} = r_t - \alpha \log \pi(a_t|s_t)$

SAC: Critic and Actor

in training updates:
expectation over states → approximated by replay buffer D
expectation over actions → sampled from current policy $\pi_{\boldsymbol{w}}$

usually min of two critics
(reduces overestimation)

critic → soft Q-function(s):

soft Q-function: $Q^{\pi}(s,a) = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k \overbrace{(r_{t+k} - \alpha \log \pi(a_{t+k}|s_{t+k}))}^{r_{\text{soft}}}]$

learned with TD target (depending on actor policy)

data from replay buffer → off-policy

sampled from current policy

actor:

policy minimizes $L_{\text{SAC}}(\boldsymbol{w}) = \mathbb{E}_{s\sim D, a\sim\pi_{\boldsymbol{w}}}[\alpha \log \pi_{\boldsymbol{w}}(a|s) - Q^{\pi}(s,a)]$

→ move policy toward actions with high Q (critic)

with trade-off: maximize Q vs. maximize entropy

important detail: reparameterization trick is used
to make the $a\sim\pi_w$ sampling differentiable

SAC: Properties

sample efficient (off-policy)

strong exploration (entropy)

works well in continuous control

but more complex than PPO:

- relies on learned soft Q-function(s) and off-policy bootstrapping
- coupled system: actor depends on critic, critic depends on actor, both change continuously
- more hyperparameters

Famous Example of Deep RL: AlphaGo

Monte Carlo tree search (heuristic, lookahead search) for move (i.e., action) selection

→ decision-time planning (advantage: focus on current state rather than full state space)

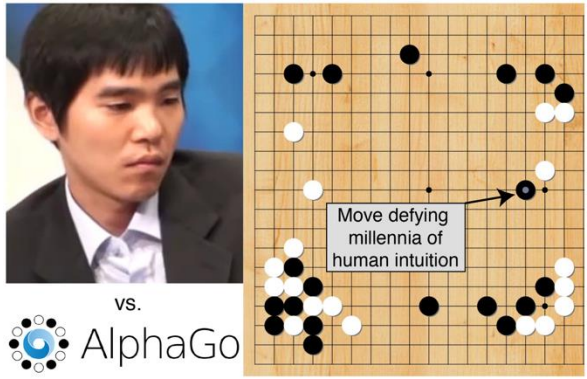
guided by deep convolutional neural networks for both value function and policy estimation

→ improving search efficiency

reduce depth of search tree by evaluating positions with **value function** (predicting outcome from given position → **bootstrapping**)

reduce breath of search tree by **sampling** actions using **policy network** (probability distribution over possible moves in given position)

famous move 37:



from Max Tegmark

Practice Project

Gymnasium Pendulum